

Tech Challenge — Fase 2 — Documento de Entrega

Curso: Pós-Tech FIAP — Arquitetura de Software (15SOAT)

Tema: Sistema Integrado de Atendimento e Execução de Serviços para Oficina Mecânica

Data de entrega: 06/07/2026

Equipe

Nome	Discord
Thiago Fernandes da Cruz	@thiago_64271

Repositório privado com acesso concedido ao grupo `soat-architecture`.

Links

Item	Link
Repositório (privado, acesso a <code>soat-architecture</code>)	https://github.com/lcr-thiago-fernandes/fiap_15SOAT_fase2
Vídeo de apresentação (≤15 min)	https://1drv.ms/f/c/fa2e7c7114d0ee1e/IgAZg-pFOF5fTJ1mQ_0nRLAmAcSgougV4IOc_4BL0KM_KVw?e=9aEKXJ
README	https://github.com/lcr-thiago-fernandes/fiap_15SOAT_fase2/blob/main/README.md
Documentos / ADRs	https://github.com/lcr-thiago-fernandes/fiap_15SOAT_fase2/tree/main/docs
Coleção de APIs	Swagger em <code>/swagger</code> + cenários em http/

Desenho da arquitetura

Os três diagramas (renderizados no README) cobrem:

1. Camadas (Clean Architecture, 5 projetos) — Entidades → Casos de Uso → Adaptadores (Controllers/Gateways/Presenters/DataSources) → Frameworks & Drivers (API/EF).

2. **Infraestrutura AWS** — VPC (2 AZs), EKS + node group, RDS PostgreSQL privado, ECR, ELB, GitHub Actions via OIDC, estado do Terraform em S3+DynamoDB.
3. **Fluxo de deploy (CI/CD)** — push em `main` → CI/CD → build/push no ECR → Job de migração → apply dos manifestos K8s → rollout.

Ver [README.md](#) e os ADRs em [docs/arquitetura](#) (ADR-009 a ADR-013 documentam as decisões da Fase 2).

Resumo da solução (Fase 2)

Evolução do MVP da Fase 1 com foco em **qualidade, resiliência e escalabilidade**, em **C# / .NET 8 / ASP.NET Core** sobre **PostgreSQL 16** (AWS RDS em produção).

O que a Fase 2 entrega

- **Refatoração para Clean Architecture** com 5 projetos, incluindo a camada `Oficina`. Adaptadores (Controllers de aplicação, Gateways, Presenters, DataSources).
- **Containerização + orquestração no Kubernetes (AWS EKS)**: Deployment com 2 réplicas, probes, `securityContext` restritivo e **HPA (2–10 réplicas)**.
- **Infraestrutura como código (Terraform)**: VPC, EKS, RDS PostgreSQL, ECR, OIDC do GitHub Actions e `metrics-server`; estado remoto em S3 + DynamoDB.
- **CI/CD (GitHub Actions)**: CI com build, testes (cobertura $\geq 80\%$), CodeQL e build da imagem; CD com deploy contínuo em `main` via **OIDC** (sem chave estática), **Job de migração** isolado e rollout controlado.
- **Novo contrato de API**: abertura consolidada da OS (cliente + veículo + serviços + peças), listagem ordenada por prioridade, **webhook de aprovação de orçamento** autenticado por token (`X-Webhook-Token` , tempo constante, fail-closed) substituindo as rotas públicas de aprovar/rejeitar (consulta pública agora é só leitura), e **notificação de status** (mock).
- **Observabilidade mínima**: logs estruturados (Serilog JSON) + métricas Prometheus em `/metrics` .

Máquina de estados da OS

Recebida → Em Diagnóstico → Aguardando Aprovação → Em Execução → Finalizada → Entregue (com **Cancelada** via recusa do orçamento no webhook).

Qualidade

- Testes unitários (Domínio/Aplicação/Adaptadores) + integração (Testcontainers, Postgres real).
- CI quebra se a cobertura de linha cair abaixo de 80%.
- CodeQL + Dependabot para análise de vulnerabilidades — ver [relatório de vulnerabilidades da Fase 2](#) (6 Altas transitivas só em teste + 1 Moderada de produção, todas aceitas com mitigação

planejada).

Como executar (resumo)

Passo a passo completo do que depende do avaliador/entregador (provisionar AWS, configurar CI/CD, deploy, demo de escalabilidade, vídeo e PDF), com justificativas amarradas ao enunciado: [GUIA-DEPLOY-E-ENTREGA-fase2.md](#).

Local (Docker):

```
cp .env.example .env # ajuste ADMIN_BOOTSTRAP_PASSWORD, JWT_SECRET, WEBHOOK_TOKEN
docker compose -f docker/docker-compose.yml --env-file .env up -d --build
# API http://localhost:8080 • Swagger /swagger • Health /health • Métricas /metrics
```

Kubernetes (AWS EKS): provisionar com Terraform (`infra/`), conectar o kubeconfig e aplicar os manifestos (`k8s/`) — ou deixar o **CD** fazer no push em `main` . Passo a passo no [README.md](#), [infra/README.md](#) e [k8s/README.md](#).

A geração do PDF a partir deste Markdown é manual (ex.: `pandoc` ou "Print to PDF").

Coleção (decisão): documentamos **Swagger** (`/swagger`) + **pasta** `http/` como a coleção (o enunciado aceita "collection ou similar"). O Postman JSON é **opcional** e **não** é gerado por este plano; se desejado no futuro, criar `docs/entrega/collection.postman_collection.json` .